NOVA IMS INFORMATION MANAGEMENT SCHOOL

Mini Test I – Solutions

Computation I / Introduction To Programming

October 24th, 2026

Version I

SOLUTIONS

Group 1

6 points: 1 point for each question.

1. The Rings of Power are numbered from 1 to 20, and rings 4 to 10 were given to the Dwarves. The user inserts the number of a ring. What should be written in place of `<a>` so that the program prints `Dwarves` for rings 4 to 10 (both included) and `someone else` for every other ring?

```
ring = int(input("Which ring? "))
if <a>:
    print("Dwarves")
else:
    print("someone else")
```

A- ✔ `ring >= 4 and ring <= 10`

B- ◯ `ring >= 4 or ring <= 10`

C- ◯ `ring >= 4 and <= 10`

D- ◯ `ring == 4 and ring == 10`

E- ◯ `ring => 4 and ring =< 10`

F- ◯ `ring > 4 and ring < 10`

G- ◯ An error occurs

H- ◯ None of the alternatives

How it works

In words, the condition is: "the ring is *at least* 4 **and** the ring is *at most* 10". Both parts must be true at the same time, so they are joined with `and`. As 4 and 10 are included, the comparisons are `>=` and `<=`. The answer is `ring >= 4 and ring <= 10`. Checking a few rings: ring 4 gives `True and True`, so `Dwarves`; ring 11 gives `True and False`, so `someone else`.

Why not the others?

- `ring >= 4 or ring <= 10`: with `or`, it is enough for *one* part to be true. Every ring is either at least 4 or at most 10, so every ring would print `Dwarves`. For example, ring 15: `15 >= 4` is `True`, so the whole condition is `True`.
- `ring >= 4 and <= 10` reads well in English, but in Python each side of `and` must be a complete comparison, so `ring` must be written again. Written like this, Python cannot even read the line, so it stops with a syntax error (the code is not valid Python).
- `ring == 4 and ring == 10` can never be true: a ring cannot be number 4 and number 10 at the same time.
- `=>` and `=<` do not exist in Python: the operators are `>=` and `<=`, with the `=` always second. Written like this, Python stops with a syntax error.
- `ring > 4 and ring < 10` leaves out rings 4 and 10 themselves.

2. Consider the following code. What will be printed on the console if the user inserts `9` (`v1`), then `98` (`v2`) and then `7` (`z`)?

```
v1 = input("How many gold dragons? ")
v2 = input("How many silver stags? ")

z = input("Gimme your fav number: ")

code = int(v1 + v2) + len(z)

if code in [42, 300, 999]:
    print(f"Vault {code} is open. A Lannister always pays his debts.")
elif code > 1000:
    print("Winter is coming.")
else:
    print("You know nothing, Jon Snow.")
```

A- ◯ `Vault 1005 is open. A Lannister always pays his debts.`
B- ◯ `Vault 108 is open. A Lannister always pays his debts.`
C- ◯ `You know nothing, Jon Snow.`
D- ◯ `Vault {code} is open. A Lannister always pays his debts.`
E- ◯ `Winter is coming.`
F- ✔ `Vault 999 is open. A Lannister always pays his debts.`
G- ◯ An error occurs
H- ◯ None of the alternatives

How it works

The key is that `input()` **always gives back text** (a string), even when the user inserts digits. So `v1` is `"9"`, `v2` is `"98"` and `z` is `"7"`.

- With two strings, `+` *joins* them instead of adding: `"9" + "98"` is `"998"`.
- `int("998")` turns that text into the number `998`. This works, with no error, because the text contains only digits.
- `len(z)` is the number of characters in `"7"`, which is `1`.
- So `code` is `998 + 1 = 999`. As 999 is in the list `[42, 300, 999]`, the first branch runs, and the f-string replaces `{code}` with 999: `Vault 999 is open. A Lannister always pays his debts.`

Why not the others?

- `Vault 1005` and `Vault 108` can never be printed: that line only runs when `code` is 42, 300 or 999.
- `You know nothing, Jon Snow.` comes from adding the inputs as numbers (`9 + 98 + 1` is 108: not in the list, not over 1000). But strings are joined, not added.
- `Vault {code}` is what would be printed without the `f` before the quotes. With the `f`, `{code}` is replaced by its value.
- `Winter is coming.` comes from adding the number typed in `z` instead of its length (`998 + 7` is 1005, over 1000). But `len(z)` counts the characters of `"7"`: 1.

3. The user inserts an integer number. What should be written in place of `<a>` so that the program prints `even` when the number is even and `odd` when it is odd?

```
n = int(input("Gimme an integer number: "))
if <a>:
    print("even")
else:
    print("odd")
```

A- ◯ `n / 2 == 0`

B- ✔ `n % 2 == 0`

C- ◯ `n // 2 == 0`

D- ◯ `n % 2 == 1`

E- ◯ `n ** 2 == 0`

F- ◯ `n % 2`

G- ◯ An error occurs

H- ◯ None of the alternatives

How it works

The operator `%` gives the **remainder** of a division. A number is even when the remainder of dividing it by 2 is 0, so the condition is `n % 2 == 0`. For example, `8 % 2` is `0` (even), and `7 % 2` is `1` (odd).

Why not the others?

- `n / 2 == 0` uses normal division: `8 / 2` is `4.0`, not 0. Only when `n` is 0 would it print `even`.
- `n // 2 == 0` uses integer division (the result without the decimal part, e.g. `8 // 2` is `4` and `7 // 2` is `3`): it is 0 only for 0 and 1.
- `n % 2 == 1` is true for *odd* numbers, so the two messages would be swapped.
- `n ** 2` is a power (n^2), not a division, so `n ** 2 == 0` is true only for 0.
- `n % 2` on its own is the wrong way round. When a number is used as a condition, Python treats 0 as `False` and any other number as `True`. For an odd number the remainder is 1 (`True`), so odd numbers would print `even`; for an even number it is 0 (`False`), so even numbers would print `odd`.

4. Consider the following code. What will be printed on the console?

```
points = 70
if points > 50:
    if points > 150:
        print("Gryffindor caught the Snitch!")
else:
    print("Slytherin wins the match!")
```

A- ◯ `points`

B- ◯ `Gryffindor caught the Snitch!`

C- ✔ **Nothing is printed**

D- ◯ `70`

E- ◯ `Slytherin wins the match!`

F- ◯ `Gryffindor caught the Snitch!`
`Slytherin wins the match!`

G- ◯ An error occurs

H- ◯ None of the alternatives

How it works

In Python, **indentation decides which `if` an `else` belongs to**. Here the `else` is at the same indentation as `if points > 50`, so it belongs to that outer `if`, not to the inner one.

- `70 > 50` is `True`, so Python enters the outer block, and its `else` is skipped.
- Inside, `70 > 150` is `False`, so the `Gryffindor` line is skipped.
- The inner `if` has no `else` of its own, so there is nothing left to run: **nothing is printed**.

Why not the others?

- `points`: no line of this code prints the variable.
- `Gryffindor caught the Snitch!` needs `points > 150`, and 70 is not larger than 150.
- `70`: as with `points`, no line of this code prints the variable.
- `Slytherin wins the match!` would be printed if the `else` were indented under the inner `if` (at the same indentation as `if points > 150`). Read the indentation, not just the order of the lines.
- `Gryffindor caught the Snitch!` followed by `Slytherin wins the match!` would need both `print` lines to run. But one is inside the outer `if` and the other is in its `else`, so at most one of them can ever run.
- No error occurs: an `if` does not need an `else`, and the indentation here is valid.

5. What should be written in place of `<a>`, `<b>` and `<c>` so that `1 4 7 10 13 16 19` is printed on the console?

```
for i in range(<a>, <b>, <c>):
    print(i, end=" ")
```

A- ◯ `<a>` = 1, `<b>` = 19 and `<c>` = 3

B- ◯ `<a>` = 1, `<b>` = 20 and `<c>` = 4

C- ✔ **`<a>` = 1, `<b>` = 20 and `<c>` = 3**

D- ◯ `<a>` = 3, `<b>` = 20 and `<c>` = 1

E- ◯ `<a>` = 1, `<b>` = 3 and `<c>` = 20

F- ◯ `<a>` = 0, `<b>` = 20 and `<c>` = 3

G- ◯ An error occurs

H- ◯ None of the alternatives

How it works

`range(start, stop, step)` begins at `start`, adds `step` each time, and **stops before reaching** `stop`: the stop value itself is never included.

- The numbers begin at 1, so `start` is `1`.
- They go up by 3 (1, 4, 7, ...), so `step` is `3`.
- The last number is 19 and the next one would be 22, so `stop` must be 20, 21 or 22: each of these includes 19 and stops before 22. `20` is the option given: `range(1, 20, 3)`.

Why not the others?

- `range(1, 19, 3)` stops *before* 19, so it prints only up to 16.
- `range(1, 20, 4)` goes up by 4, not 3: it prints `1 5 9 ...`
- `range(3, 20, 1)` prints `3 4 5 ... 19`, and `range(1, 3, 20)` prints only `1`: the order is always start, stop, step.
- `range(0, 20, 3)` starts at 0, not 1: it prints `0 3 6 9 ...`

6. Consider the following code. What will be printed on the console?

```
finds = "rex-raptor-REX-bronto-rex"
lt = finds.split("-")
res = 0
for x in lt:
    if x == "rex":
        res = res + 1
print(f"{res} of {len(lt)} fossils are rex")
```

A- ◯ `3 of 5 fossils are rex`

B- ◯ `2 of 25 fossils are rex`

C- ◯ `5 of 5 fossils are rex`

D- ◯ `{res} of {len(lt)} fossils are rex`

E- ◯ `0 of 1 fossils are rex`

F- ✔ `2 of 5 fossils are rex`

G- ◯ An error occurs

H- ◯ None of the alternatives

How it works

Follow the code line by line:

- `split("-")` cuts the string at every `-` and gives a list: `['rex', 'raptor', 'REX', 'bronto', 'rex']`. It has 5 items, so `len(lt)` is 5.
- The loop adds 1 to `res` for each item that is *exactly* `"rex"`. Comparing strings is **case-sensitive** (capital and small letters count as different): `"REX" == "rex"` is `False`. Only the first and the last items count, so `res` is 2.
- The output is `2 of 5 fossils are rex`.

Why not the others?

- `3 of 5` would treat `REX` as `rex`. To count `REX` too, the condition would need to be `x.lower() == "rex"`.
- `2 of 25`: 25 is the number of characters in the original string `finds`, but `len(lt)` counts the items of the list.
- `5 of 5` counts every item, but the `if` adds 1 only when the item is exactly `"rex"`.
- `{res} of ...` would only be printed without the `f` before the quotes.
- `0 of 1` is what you would get if `split` did not cut the string: a list with only one item, the whole string, which is not `"rex"`. But the string is cut at every `-`, so the list has 5 items.

SOLUTIONS

Group 2

4 points: 2 points for each question.

NOTE: Each question in this group shows a block of code and states what that code was supposed to do. There is an issue with each block of code: it does not do what was intended. Your task is to find the issue and fix it, as follows:

A State the issue briefly on the line after "What is the issue?".

B Rewrite each faulty line of code, corrected, on the empty line directly below it. Leave all the other empty lines blank.

Example of a complete answer. The answers are written in blue, and the letters in the margin show where each step was answered. In the solutions that follow, the answers are written in green.

This block of code was supposed to print `yayyy yummy!` when the favourite fruit of the user is grapefruit, however it is capitalised.

A **What is the issue?** *print() returns nothing; () missing after lower*

Correctly identify and **fix the problematic line(s)** below.

```
1 fruit = print(input("Gimme your favourite fruit?"))
```

B *fruit = input("Gimme your favourite fruit?")*

```
2
3 if fruit.lower == "grapefruit":
```

B *if fruit.lower() == "grapefruit":*

```
4     print("yayyy yummy!")
```

The questions of this group start on the next page.

SOLUTIONS

1. This block of code was supposed to warn when the first number of the list is smaller than 5 and, otherwise, to say whether the list has more than three numbers.

What is the issue? `lt[0]` is the number 10. `len()` counts the items of a list or the characters of a string, and a single number has neither, so line 3 stops with an error (`TypeError`). To compare the first number with 5, use `lt[0]` directly.

Correctly identify and **fix the problematic line(s)** below.

```
1 lt = [10, 20, 30]
2
3 if len(lt[0]) < 5:
  if lt[0] < 5:
4     print(f"warning! first number too small")
5 elif len(lt) > 3:
6     print(f"{lt} has more than 3 elements, all good!")
7 else:
8     print(f"{lt} has 3 elements or fewer, ew...")
```

2. This block of code was supposed to create the Pawnee Parks and Recreation email of each member of the staff and store them in a list. When the code runs, exactly the following was supposed to be printed on the console, and nothing else:
`['leslie@pawnee.gov', 'ron@pawnee.gov', 'april@pawnee.gov']`

What is the issue? The `print` on line 5 is indented inside the `for` loop, so it runs after every name and prints the list three times while it is still growing: first with one email, then two, then three. At the same indentation as `for`, it runs once, after the loop, and prints the finished list.

Correctly identify and **fix the problematic line(s)** below.

```
1 staff = ["Leslie", "Ron", "April"]
2 res = []
3 for x in staff:
4     res.append(x.lower() + "@pawnee.gov")
5     print(res)
print(res)
```

SOLUTIONS

Group 3

2 points

NOTE: This group corresponds to direct questions. Please answer the questions within the space provided for the answer. The amount of space provided is just speculative. Do not feel inclined to add more just to fill in the space.

1. The string `family` below contains the names of the Griffin family, separated by semicolons. What needs to be written instead of `<A>`, `<B>`, `<C>`, `<D>` and `<E>` so that the program works as described below?

   - The string is split into a list with one name in each position (`<A>`).
   - The loop goes through every name of that list (`<B>` and `<C>`).
   - Each name with more than four letters (`<D>`) is added to `res` in capital letters (`<E>`).

   In the end, the program prints `['PETER', 'CHRIS', 'STEWIE']`.

```
family = "peter;lois;meg;chris;stewie"
lt = family.split(<A>)
res = []

for <B> in <C>:
    if <D>:
        res.append(<E>)

print(res)
```

(0.4 points `<A>`): `";"`

(0.4 points `<B>`): `x` (any variable name)

(0.4 points `<C>`): `lt`

(0.4 points `<D>`): `len(x) > 4`

(0.4 points `<E>`): `x.upper()`

How it works

The complete program is:

```
family = "peter;lois;meg;chris;stewie"
lt = family.split(";")
res = []

for x in lt:
    if len(x) > 4:
        res.append(x.upper())

print(res)
```

- `<A>`: `family.split(";")` cuts the string at every semicolon and gives the list `['peter', 'lois', 'meg', 'chris', 'stewie']`. The separator is text, so it goes in quotes: `";"`.
- `<B>`: `x` is the variable that holds the current name in each round of the loop. You choose this variable's name (`name` or `person` work too), but you must then use the same one in `<D>` and `<E>`.
- `<C>`: the loop must go through the list `lt`. Looping over `family` instead would go through the string one *character* at a time.
- `<D>`: `len(x) > 4` means "more than four letters" (`len(x) >= 5` is the same). `peter` (5), `chris` (5) and `stewie` (6) pass; `lois` has exactly 4 letters, which is not *more* than 4, and `meg` has 3.
- `<E>`: `x.upper()` gives the name in capital letters, and `res.append(...)` adds it to the end of `res`. The round brackets `()` matter: without them, `x.upper` does not change the name into capital letters.

SOLUTIONS

Group 4

8 points

NOTE: This group corresponds to open questions. Make sure you present all necessary code when code is requested from you. Feel free to write notes and other things to show your thought process when you feel like it may be necessary.

Teddy, the most loyal customer of Bob's Burgers, has just sat down at the counter. The program starts with the following two lines of code, which are already written:

```
menu = ["bacon", "cheese", "teriyaki", "mushroom", "pineapple", "beets"]
orders = ["fries", "coleslaw"]
```

Continue the program, answering each of the following questions below the previous one.

1. **(5 points)** Write the code that does the following:

   a) Print `Today's menu is:` followed by the menu, one burger per line.

   b) Ask Teddy which burger he would like.

   c) If that burger is on the menu (regardless of the capitalisation Teddy used to order it), add it to Teddy's orders and print a message with Teddy's full order saying that it is coming right up. Otherwise, print a message saying that it is not on the menu.

   Example of a correct run of the program (what Teddy inserts is in grey; your messages may be worded differently):

   ```
   Today's menu is:
   bacon
   cheese
   teriyaki
   mushroom
   pineapple
   beets
   What burger would you like, Teddy? BEETS
   Teddy's order: ['fries', 'coleslaw', 'beets']. Coming right up!
   ```

Solution

```
# a) Print the title, then go through the menu
#    and print each burger on a line of its own
#    (print(menu) would show the whole list on one
#    line, with brackets)
print("Today's menu is:")
for x in menu:
    print(x)

# b) input() always gives back text. .lower() puts
#    it in small letters, so "BEETS", "Beets" and
#    "beets" all become "beets", which is how the
#    burgers are written in menu.
#    Careful: without the (), .lower does not change the text
burger = input("What burger would you like, Teddy? ").lower()

# c) "in" checks whether burger is one of the items
#    of the list menu
if burger in menu:
    # append() adds one item to the end of a list
    # (orders + burger would not work: a list and
    # a string cannot be added together)
    orders.append(burger)
    print(f"Teddy's order: {orders}. Coming right up!")
else:
    print(f"Sorry Teddy, there is no {burger} burger on the menu.")
```

2. **(3 points)** Teddy's fries and coleslaw are part of a combo that costs 5 euros. All burgers cost 8.99 euros, but every day, the last burger on the menu has a 10% discount. Calculate the total cost of Teddy's order, add 23% tax to it and print the final total, rounded to two decimal places. In this question, you can assume that the burger Teddy ordered is on the menu. **Your code must work for any day, even if the menu changes.**

   Example of a correct run of the program, continuing the example of the previous question (Teddy ordered the beets burger, which happens to be the last one on today's menu. Your message may be worded differently as long as it conveys the intended message):

   ```
   With tax, Teddy's total is 16.1 euros.
   ```

   Solution

```python
# Start with the price of the combo (fries + coleslaw)
total = 5

# burger was saved in the previous question, already
# in small letters.
# Negative positions count from the end: menu[-1] is
# the last item, whatever today's menu is. Writing
# "beets" instead would only work today, and the code
# must work for any day.
# (menu[len(menu) - 1] is also the last item)
if burger == menu[-1]:
    # 10% off means paying 90% of the price
    total = total + 8.99 * 0.9
else:
    total = total + 8.99

# The 10% discount is only on the burger, not on the
# combo, so it is simplest to apply it first (above).
# The 23% tax goes on the whole order: adding 23%
# means multiplying by 1.23 (100% + 23% more).
# round(..., 2) rounds to two decimal places; a final
# 0 is not shown (16.10 prints as 16.1). Any other way
# of rounding to two decimal places is also fine.
total = round(total * 1.23, 2)
print(f"With tax, Teddy's total is {total} euros.")
```